Prompt Templates

1. Introduction

In generative AI applications, prompts are often constructed repeatedly for similar tasks. For example, an application may need to generate a summary for hundreds of different pieces of text. Writing the complete prompt manually for every input is inefficient and makes the application difficult to maintain. **Prompt templates** solve this problem by providing a reusable structure in which changing information can be inserted dynamically.

A prompt template separates the **fixed instructions** from the **variable information**. Instead of creating a completely new prompt every time, a developer defines the prompt structure once and supplies different values whenever the application runs. Prompt templates are particularly useful when building LLM-based applications because they make prompts reusable, consistent, easier to maintain, and easier to integrate into larger workflows.

---

2. What is a Prompt Template?

A **prompt template** is a predefined structure for creating prompts dynamically by combining fixed instructions with variable values.

Consider the following prompt:

"Explain Python programming to a beginner."

If we want to use the same structure for different topics, manually changing the topic every time is inconvenient.

Instead, we can create a template:

"Explain {topic} to a beginner."

Here, {topic} is a **variable**.

When the value Python programming is supplied, the template becomes:

"Explain Python programming to a beginner."

If the value is changed to Machine Learning, the same template produces:

"Explain Machine Learning to a beginner."

Therefore, a prompt template can be represented conceptually as:

Prompt Template + Input Values → Final Prompt

---

3. Components of a Prompt Template

A prompt template generally contains two important parts.

3.1 Fixed Instructions

Fixed instructions are the parts of the prompt that remain unchanged across different executions.

For example:

"Explain the following topic in simple English."

This instruction can remain the same regardless of the topic.

3.2 Variables

Variables represent information that changes from one execution to another.

For example:

{topic}

The value of {topic} could be:

- Python
- Machine Learning
- Generative AI
- Natural Language Processing

The template therefore becomes reusable for multiple inputs.

---

4. Why Use Prompt Templates?

Prompt templates provide several advantages when developing generative AI applications.

Reusability

A single template can be used with many different inputs instead of creating separate prompts.

Consistency

The same instructions are applied to every input, producing a more consistent prompting structure.

Maintainability

If the prompt needs to be modified, the developer can change the template rather than modifying every individual prompt.

Dynamic Input

Variables allow user-provided or application-generated information to be inserted into the prompt dynamically.

Application Development

Prompt templates are particularly useful when prompts are part of larger LLM applications, pipelines, or automated workflows.

---

5. Prompt Templates in LangChain

LangChain provides the **PromptTemplate** class for creating reusable prompts.

It can be imported using:

```
from langchain_core.prompts import PromptTemplate
```

A basic prompt template can be created by specifying a template string containing variables.

For example:

```
prompt = PromptTemplate.from_template("Explain {topic} in simple English.")
```

Here, {topic} is a variable.

A value can then be supplied to the template:

```
prompt.invoke({"topic": "Generative AI"})
```

The resulting prompt becomes:

Explain Generative AI in simple English.

---

6. Using PromptTemplate with an LLM

A prompt template becomes more useful when it is connected to a language model.

In LangChain, the prompt can be passed to a model using a pipeline such as:

PromptTemplate → Chat Model → Response

The prompt template generates the final prompt, and the chat model processes that prompt to generate the response.

For local models, LangChain can use **Ollama** as the model provider. In the following example, the gemma3:270m model is used.

---

7. Sample Code Using PromptTemplate and Ollama

```
from langchain_core.prompts import PromptTemplate

from langchain.chat_models import init_chat_model


model = init_chat_model("gemma3:270m", model_provider="ollama")


prompt = PromptTemplate.from_template("Explain {topic} in simple English with one real-world example.")


chain = prompt | model


response = chain.invoke({"topic": "Generative AI"})
```

```
print(response.content)
```

How the Code Works

The following line imports PromptTemplate:

```
from langchain_core.prompts import PromptTemplate
```

The following line imports LangChain's model initialization function:

```
from langchain.chat_models import init_chat_model
```

The model is initialized using the Ollama provider:

```
model = init_chat_model("gemma3:270m", model_provider="ollama")
```

The prompt template contains a variable called {topic}:

```
prompt = PromptTemplate.from_template("Explain {topic} in simple English with one real-world example.")
```

The prompt and model are connected using the LangChain pipe operator:

```
chain = prompt | model
```

When the chain is invoked:

```
response = chain.invoke({"topic": "Generative AI"})
```

the value "Generative AI" replaces {topic} in the template.

The model therefore receives a prompt equivalent to:

"Explain Generative AI in simple English with one real-world example."

Finally:

```
print(response.content)
```

prints the generated response.

---

8. Multiple Variables in a Prompt Template

A prompt template can contain multiple variables.

For example:

```
prompt = PromptTemplate.from_template("Explain {topic} to a {audience} using a {style} style.")
```

The variables can be supplied together:

```
response = chain.invoke({"topic": "Transformers", "audience": "beginner", "style": "simple"})
```

The resulting prompt becomes:

Explain Transformers to a beginner using a simple style.

This makes prompt templates suitable for applications where several parts of the prompt change dynamically.

---

9. Prompt Template Workflow

The overall process can be understood as:

```
Create Prompt Template
        ↓
Define Variables
        ↓
Provide Variable Values
        ↓
Generate Final Prompt
        ↓
Send Prompt to Chat Model
        ↓
Receive AI Response
```

Thus, a prompt template acts as a **reusable bridge between application inputs and the language model**.

---

10. Key Takeaway

A **PromptTemplate** allows developers to define a reusable prompt structure containing variables that can be dynamically replaced with actual values.

The basic idea is:

Template + Variables → Final Prompt → LLM → Response

When combined with LangChain and an Ollama-hosted model such as gemma3:270m, prompt templates provide a simple foundation for building reusable local generative AI applications.